CS 224n 2024年春季：作业 #3

截止日期：4月30日，星期二，下午4:30（太平洋标准时间）。
本作业分为两个部分：使用*RNN*的神经机器翻译和分析*NMT*系统。第一部分主要侧重于编码和实现，而第二部分完全由书面分析问题组成。如果你在第一部分遇到困难，可以随时先做第二部分，因为这两个部分是相互独立的。请注意，NMT系统比我们在本课程中之前构建的神经网络更为复杂，在GPU上训练大约需要**2小时**。因此，我们强烈建议你尽早开始本作业。最后，NMT系统的符号表示和实现有些棘手，所以如果你在过程中遇到困难，请来参加答疑时间，以便助教能够为你提供支持。

1. 使用RNN的神经机器翻译（45分）

在机器翻译中，我们的目标是将一个句子从源语言（例如普通话中文）转换为目标语言（例如英语）。在本作业中，我们将实现一个带有注意力机制的序列到序列（Seq2Seq）网络，以构建一个神经机器翻译（NMT）系统。在本节中，我们描述所提出的NMT系统的**训练过程**，该系统使用双向LSTM编码器和单向LSTM解码器。

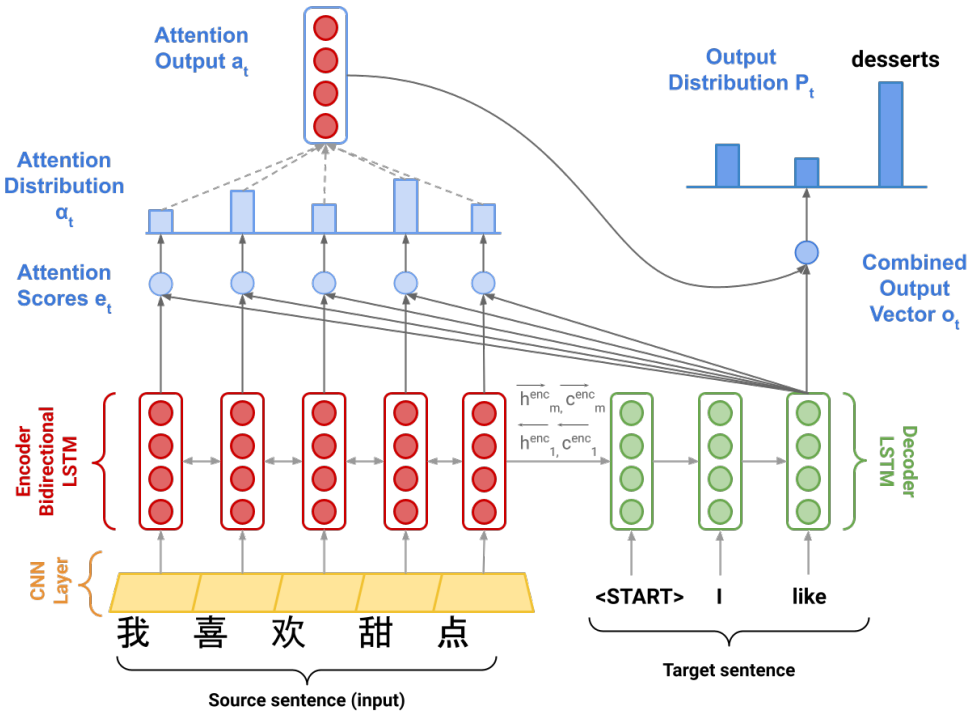


图1：带乘法注意力的Seq2Seq模型，展示在解码器的第三步。隐藏状态h_i^{enc}和单元状态c_i^{enc}在下一页定义。

模型描述（训练过程）

给定源语言中的一个句子，我们从**嵌入矩阵**中查找字符或词嵌入，得到 $x_1, \ldots, x_m$ ($x_i \in \mathbb{R}^{e \times 1}$)，其中 m 是源句子的长度， e 是

嵌入维度。然后我们将嵌入输入到**卷积层** [1]，同时保持其形状不变。我们将卷积层的输出输入到**双向编码器**，得到前向（→）和后向（←）LSTM 的隐藏状态和细胞状态。前向和后向版本被拼接起来，得到隐藏状态 $\mathbf{h}_i^{\text{enc}}$和细胞状态 $\mathbf{c}_i^{\text{enc}}$:

$$\mathbf{h}_i^{\text{enc}} = [\overleftarrow{\mathbf{h}_i^{\text{enc}}}; \overrightarrow{\mathbf{h}_i^{\text{enc}}}] \text{ where } \mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h\times 1}, \overleftarrow{\mathbf{h}_i^{\text{enc}}}, \overrightarrow{\mathbf{h}_i^{\text{enc}}} \in \mathbb{R}^{h\times 1} \qquad 1 \le i \le m \tag{1}$$

$$\mathbf{c}_i^{\text{enc}} = [\overleftarrow{\mathbf{c}_i^{\text{enc}}}; \overrightarrow{\mathbf{c}_i^{\text{enc}}}] \text{ where } \mathbf{c}_i^{\text{enc}} \in \mathbb{R}^{2h\times 1}, \overleftarrow{\mathbf{c}_i^{\text{enc}}}, \overrightarrow{\mathbf{c}_i^{\text{enc}}} \in \mathbb{R}^{h\times 1} \qquad 1 \le i \le m \tag{2}$$

然后我们用编码器最终隐藏状态和最终单元状态的线性投影来初始化**解码器**的第一个隐藏状态 $\mathbf{h}_0^{\text{dec}}$ 和单元状态 $\mathbf{c}_0^{\text{dec}}$。[2]

$$\mathbf{h}_0^{\text{dec}} = \mathbf{W}_h[\overleftarrow{\mathbf{h}_1^{\text{enc}}}; \overrightarrow{\mathbf{h}_m^{\text{enc}}}] \text{ where } \mathbf{h}_0^{\text{dec}} \in \mathbb{R}^{h\times 1}, \mathbf{W}_h \in \mathbb{R}^{h\times 2h} \tag{3}$$

$$\mathbf{c}_0^{\text{dec}} = \mathbf{W}_c[\overleftarrow{\mathbf{c}_1^{\text{enc}}}; \overrightarrow{\mathbf{c}_m^{\text{enc}}}] \text{ where } \mathbf{c}_0^{\text{dec}} \in \mathbb{R}^{h\times 1}, \mathbf{W}_c \in \mathbb{R}^{h\times 2h} \tag{4}$$

解码器初始化完成后，我们需要向其输入一个目标句子。在第 t^{th} 步，我们查找第 t^{th} 个子词的嵌入，$\mathbf{y}_t \in \mathbb{R}^{e\times 1}$。然后我们将 y_t 与上一时间步的组合输出向量$\mathbf{o}_{t-1} \in \mathbb{R}^{h\times 1}$ 拼接（我们将在本页后面解释这是什么！），得到 $\mathbf{y}_t \in \mathbb{R}^{(e+h)\times 1}$。注意，对于第一个目标子词（即起始标记），$\mathrm{o}_0$ 是一个零向量。然后我们将 y_t 作为输入馈送给解码器。

$$\mathbf{h}_t^{\text{dec}}, \mathbf{c}_t^{\text{dec}} = \text{Decoder}(\overline{\mathbf{y}_t}, \mathbf{h}_{t-1}^{\text{dec}}, \mathbf{c}_{t-1}^{\text{dec}}) \text{ where } \mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h\times 1}, \mathbf{c}_t^{\text{dec}} \in \mathbb{R}^{h\times 1} \tag{5}$$

$$\tag{6}$$

然后我们使用 $\mathbf{h}_t^{\text{dec}}$ 来计算对 $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_m^{\text{enc}}$: 的乘法注意力

$$\mathbf{e}_{t,i} = (\mathbf{h}_t^{\text{dec}})^T \mathbf{W}_{\text{attProj}} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{e}_t \in \mathbb{R}^{m\times 1}, \mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h\times 2h} \qquad 1 \le i \le m \tag{7}$$

$$\alpha_t = \text{softmax}(\mathbf{e}_t) \text{ where } \alpha_t \in \mathbb{R}^{m\times 1} \tag{8}$$

$$\mathbf{a}_t = \sum_{i=1}^{m} \alpha_{t,i} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{a}_t \in \mathbb{R}^{2h\times 1} \tag{9}$$

$\mathrm{e}_{t,i}$ 是一个标量，即 $\mathbf{e}_t \in \mathbb{R}^{m\times 1}$ 的第 i 个元素，使用解码器在第 t 步的隐藏状态 $\mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h\times 1}$、注意力投影 $\mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h\times 2h}$ 以及编码器在第 i 步的隐藏状态 $\mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h\times 1}$ 计算得到。

我们现在将注意力输出 a_t 与解码器隐藏状态 $\mathbf{h}_t^{\text{dec}}$拼接，并将其通过一个线性层、tanh 和 dropout，以获得 组合输出向量 o_t。

$$\mathbf{u}_t = [\mathbf{a}_t; \mathbf{h}_t^{\text{dec}}] \text{ where } \mathbf{u}_t \in \mathbb{R}^{3h\times 1} \tag{10}$$

$$\mathbf{v}_t = \mathbf{W}_u \mathbf{u}_t \text{ where } \mathbf{v}_t \in \mathbb{R}^{h\times 1}, \mathbf{W}_u \in \mathbb{R}^{h\times 3h} \tag{11}$$

$$\mathbf{o}_t = \text{dropout}(\tanh(\mathbf{v}_t)) \text{ where } \mathbf{o}_t \in \mathbb{R}^{h\times 1} \tag{12}$$

[1]查看 https://cs231n.github.io/convolutional-networks 以获取卷积层的深入描述，如果你还不熟悉的话[2]如果这还不明显，想一想为什么我们将 $[\overleftarrow{\mathbf{h}_1^{\text{enc}}}, \overrightarrow{\mathbf{h}_m^{\text{enc}}}]$ 视为编码器的“最终隐藏状态”。

然后，我们在第 t^{th} 个时间步生成目标子词上的概率分布 P_t：

$$\mathbf{P}_t = \text{softmax}(\mathbf{W}_{\text{vocab}}\mathbf{o}_t) \text{ where } \mathbf{P}_t \in \mathbb{R}^{V_t \times 1}, \mathbf{W}_{\text{vocab}} \in \mathbb{R}^{V_t \times h} \tag{13}$$

这里， V_t 是目标词汇表的大小。最后，为了训练网络，我们计算 P_t 和 g_t之间的交叉熵损失，其中 g_t 是时间步 t 处目标子词的单热向量：

$$J_t(\theta) = \text{CrossEntropy}(\mathbf{P}_t, \mathbf{g}_t) \tag{14}$$

这里， θ 表示模型的所有参数， J_t（θ）是解码器第 t 步的损失。既然我们已经描述了该模型，让我们尝试将其实现用于中文到英文的翻译！

设置本地开发环境

确保您已安装 conda 。解压起始代码，并在您喜欢的 Python 集成开发环境中打开生成的目录（ Visual Studio Code 是一个热门选择）。打开终端并导航（cd）到包含 env-cpu.yml 和 env-gpu.yml 文件的目录。按如下方式创建并激活 cs224n-cpu conda 环境：

```
conda env create --file env-cpu.yml
conda activate cs224n-cpu
```

设置你的云GPU驱动的虚拟机

按照CS224n的GCP指南（链接也提供在网站和Ed上）中的说明来创建你的虚拟机实例。这大约需要45分钟。尽管你需要GPU来训练模型，但我们强烈建议你先在本地开发代码并确保其能运行，然后再尝试在虚拟机上训练。GPU时间昂贵且有限。训练NMT系统大约需要**1.5到2小时**。我们不希望你意外地将所有GPU时间用于调试模型，而不是训练和评估它。最后，**确保在不使用虚拟机时将其关闭**。

如果你的GCP订阅资金用完，你的虚拟机将被暂时锁定且无法访问。如果发生这种情况，请发布一条私密的Ed帖子描述情况。
设置好云虚拟机后，你在本地完成编程作业期间应将其关闭。

实现和书面问题

(a)（2分）（编码）为了应用张量操作，我们必须确保给定批次中的句子长度相同。因此，我们必须找出批次中最长的句子，并将其他句子填充到相同长度。在utils.py中实现pad_sents函数，该函数应生成这些填充后的句子。

(b)（3分）（编程）在 model_embeddings.py 中实现 __init__ 函数，以初始化必要的源嵌入和目标嵌入。
(c)（4分）（编程）在 nmt_model.py 中实现 __init__ 函数，以初始化 NMT 系统所需的模型层（LSTM、CNN、投影和 dropout）。

(d)（8分）（编程）在nmt_model.py中实现encode函数。该函数将填充后的源句子转换为张量X，生成 $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_m^{\text{enc}}$，并计算解码器的初始状态$\mathbf{h}_0^{\text{dec}}$和初始单元 $\mathbf{c}_0^{\text{dec}}$。你可以通过执行以下命令运行一个非全面的健全性检查：

```
python sanity_check.py 1d
```

(e)（8分）（编程）在nmt_model.py中实现decode函数。该函数构造 ¯y，并对输入的每个时间步运行step函数。你可以通过执行以下命令运行一个非全面的健全性检查：

```
python sanity_check.py 1e
```

(f)（10分）（编程）在 nmt_model.py 中实现 step 函数。该函数对单个时间步应用解码器的 LSTM 单元，计算目标子词 h_t^{dec} 的编码、注意力分数 e_t、注意力分布 α_t、注意力输出 a_t，以及最终组合输出 o_t。你可以通过执行以下命令来运行一个非全面的健全性检查：

```
python sanity_check.py 1f
```

(g)（3分）（书面）nmt_model.py中的generate_sent_masks()函数会生成一个名为enc_masks的张量。它的形状为（批量大小，最大源句子长度），在对应输入中'pad'标记的位置包含1，非pad标记的位置包含0。看看这些掩码是如何

在 step() 函数中的注意力计算过程中使用（第 295-296 行）。

首先解释（大约三句话）掩码对整个注意力计算的影响。然后解释（一到两句话）为什么有必要以这种方式使用掩码。

现在是时候让它运行起来了！如前所述，我们建议你在个人电脑上开发代码。确认你正在正确的 conda 环境中运行，然后执行以下命令在本地机器上训练模型：

```
sh run.sh train_local
(Windows) run.bat train_local
```

为了帮助监控和调试，起始代码使用 tensorboard 在训练期间通过 TensorBoard 记录损失和困惑度[3]。TensorBoard 提供了用于记录和可视化实验训练信息的工具。要打开 TensorBoard 并监控训练过程，请在你同样已激活 cs224n-cpu conda 环境的另一个终端中运行以下命令，然后在 http://localhost:6006/ 访问 tensorboard。

```
tensorboard --logdir=runs --port 6006
```

你应该会看到在初始迭代期间损失显著下降（图 2）。一旦你的代码运行了几百次迭代而没有崩溃，就从 GCP 控制台启动你的虚拟机。

在虚拟机上训练 NMT 系统：请参阅 GCP 操作指南附录以：

- 设置您的 GCP 虚拟机并获取 SSH 连接信息
- 连接到虚拟机（设置 SSH 隧道，以便您可以查看远程 TensorBoard 日志）
- 将您的代码从本地计算机复制到云端虚拟机
- 在云端虚拟机上训练 NMT 系统
- 从您的云端虚拟机下载 Gradescope 提交包

[3]https://pytorch.org/docs/stable/tensorboard.html

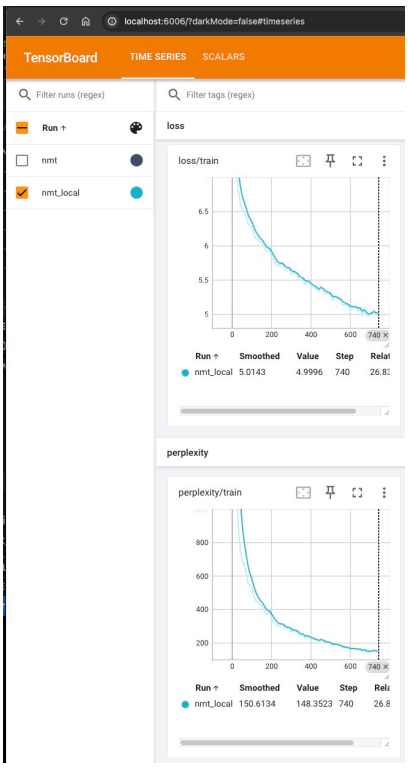


图2：在本地机器上显示损失值和困惑度值的Tensorboard

(h)（3 分）（书面）模型训练完成后（**在虚拟机上这应该需要不到 2 小时**），执行以下命令来测试模型：

```
sh run.sh test
```

请报告模型的语料库BLEU分数。它应该大于18。

(i)（4分）（书面）在课堂上，我们学习了点积注意力、乘法注意力和加法注意力。提醒一下，点积注意力是$\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{h}_i$，乘法注意力是$\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{W} \mathbf{h}_i$，加法注意力是$\mathbf{e}_{t,i} = \mathbf{v}^T \tanh(\mathbf{W}_1 \mathbf{h}_i + \mathbf{W}_2 \mathbf{s}_t)$。

i.（2分）解释点积注意力与乘法注意力相比的一个优点和一个缺点。

ii.（2分）解释加性注意力与乘法注意力相比的一个优点和一个缺点。

2. 分析NMT系统（25分）

(a)（3分）查看 `src.vocab` 文件，了解源语言词汇表中短语和单词的一些示例。在将输入的普通话中文序列编码为词汇表中的“片段”时，分词器会将序列映射为一系列词汇项，每个词汇项由一个或多个字符组成（得益于 `sentencepiece` 分词器，即使原始文本没有空格，我们也能进行这种切分）。基于这些信息，在嵌入层之后、将嵌入传入双向编码器之前添加一个一维卷积层，会如何帮助我们的NMT系统？**提示：**每个普通话中文字符要么是一个完整的词，要么是词中的一个语素。分别查阅电、脑和电脑的含义作为示例。字符电（electricity）和脑（brain）组合成短语电脑时，意为computer。

(b)（8分）这里我们展示在NMT模型（与你刚刚训练的模型相同）输出中发现的一系列错误。对于每一个参考（即“gold”）英文翻译和NMT（即“model”）英文翻译的示例，请：

1. 找出NMT翻译中的错误。
2. 提供模型可能出错的原因（可能是由于特定的语言结构或特定的模型局限性）。

3. 描述一种可能修改NMT系统以修正所观察到错误的方法。一个错误可能有不止一种修正方法。例如，可以调整隐藏层的大小或改变注意力机制。

以下是你需要按照上述要求进行分析的翻译。只需分析每个句子中带下划线的错误。请放心，你不需要懂中文就能回答这些问题。你只需要懂英语！不过，如果你想对源句子有更多了解，可以随意使用https://www.archchinese.com/chinese_english_dictionary.html 这样的资源来查词。也可以随意搜索训练数据文件，以更好地了解某些字符出现的频率。

i.（2分）**源句：**贼人其后被警方拘捕及被判处盗窃罪名成立。
参考译文：*the culprits were subsequently arrested and convicted.*
NMT译文：*the culprit was subsequently arrested and sentenced to theft.*

ii.（2分）**源句**：几乎已经没有地方容纳这些人，资源已经用尽。
参考译文：*there is almost no space to accommodate these people, and resources have run out.*
NMT翻译：资源已经耗尽，资源已经耗尽。

iii.（2分）**源句**：当局已经宣布今天是国殇日。
参考译文：当局已宣布今天为全国哀悼日。
NMT翻译：政府已经宣布了今天。

iv.（2分）**源句**[4]**：**俗语有云：“唔做唔错”。
参考译文：“不做不错”，正如俗话所说。
NMT翻译：俗话说，“这并没有错。”

(c)（14分）BLEU分数是NMT系统最常用的自动评估指标。
它通常在整个测试集上计算，但这里我们将考虑为单个样本定义的BLEU。[5]假设我们有一个源句子$\mathbf{s}$、一组 k 参考译文$\mathbf{r}_1, \ldots, \mathbf{r}_k$，以及一个候选译文$\mathbf{c}$。为了计算$\mathbf{c}$的BLEU分数，我们首先计算修正的 n-*gram*精确率 p_n，针对$\mathbf{c}$，对于每个 $n = 1, 2, 3, 4$，其中 n 是 n 在n-gram中：

$$p_n = \frac{\sum_{\text{ngram} \in \mathbf{c}} \min\left(\max_{i=1,\ldots,k} \text{Count}_{\mathbf{r}_i}(\text{ngram}),\ \text{Count}_{\mathbf{c}}(\text{ngram}) \right)}{\sum_{\text{ngram} \in \mathbf{c}} \text{Count}_{\mathbf{c}}(\text{ngram})} \tag{15}$$

在此，对于候选译文 c 中出现的每个 n-gram，我们统计它在任意一个参考译文中出现的最大次数，并以它在 c 中出现的次数为上限（此为分子）。我们将此除以 c 中 n-gram 的数量（分母）。

[4]这是一个粤语句子！本作业使用的数据来自GALE Phase 3，它是从互联网上抓取的各种来源的简体中文新闻及其译文的汇编。更多详情，请参见https://catalog.ldc.upenn.edu/LDC2017T02。[5]这种句子级BLEU分数的定义与nltk Python包中的句子_bleu()函数一致。请注意，NLTK函数对大小写敏感。在本问题中，所有文本均为小写，因此大小写无关紧要。
http://www.nltk.org/api/nltk.translate.html#nltk.translate.bleu_score.sentence_bleu

接下来，我们计算 简洁性惩罚 BP。设 $len(c)$ 为 c 的长度，并设 $len(r)$ 为最接近 $len(c)$ 的参考译文长度（若有两个参考译文长度同样接近，则选择 $len(r)$ 作为较短的那个）。

$$BP = \begin{cases} 1 & \text{if } len(c) \geq len(r) \\ \exp\left(1 - \frac{len(r)}{len(c)}\right) & \text{otherwise} \end{cases} \tag{16}$$

最后，候选译文 c 相对于 $\mathrm{r}_1, \ldots, \mathrm{r}_k$ 的 BLEU 分数为：

$$BLEU = BP \times \exp\left(\sum_{n=1}^{4} \lambda_n \log p_n\right) \tag{17}$$

其中 $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ 是总和为 1 的权重。此处的 log 为自然对数。

i.（5 分）请考虑以下示例：
源句子 s：**需要有充足和可预测的资源**。
参考译文 r_1：资源必须充足且可预测参考译文 r_2：需要充足且可预测的资源NMT 译文 c_1：需要充足且可预测的资源

NMT 译文 c_2：资源应充足且可预测，以便
请计算 c_1 和 c_2 的 BLEU 分数。 令 $\lambda_i = 0.5$ 为 $i \in \{1, 2\}$， $\lambda_i = 0$ 为 $i \in \{3, 4\}$（**这意味着我们忽略 3-gram 和 4-gram**，即不计算 p_3 或 p_4）。计算 BLEU 分数时，请展示你的计算过程（即展示你计算出的 p_1、 p_2、len（c）、 len（r）和 BP 的值）。注意，BLEU 分数可以表示为 0 到 1 之间，或
0 到 100 之间。代码使用的是 0 到 100 的尺度，而在这个问题中我们使用的是
0到1量表。请将您的回答四舍五入到3位小数。

根据BLEU分数，这两个NMT翻译中哪一个被认为是更好的翻译？您是否同意它是更好的翻译？

ii.（5分）我们的硬盘损坏了，丢失了参考译文r_1。请仅针对r_2 重新计算c_1 和c_2, 的BLEU分数。现在这两个NMT翻译中哪一个获得更高的BLEU分数？您是否同意它是更好的翻译？

iii.（2分）由于数据可用性的限制，NMT系统通常仅针对单个参考译文进行评估。请解释（用几句话）为什么这可能存在问题。在您的解释中，请讨论当存在多个参考译文与单个参考译文时，BLEU分数指标如何评估NMT翻译的质量。

iv.（2分）与人工评估相比，列出BLEU作为机器翻译评估指标的两个优点和两个缺点。

(d)（4分）束搜索常用于提升机器翻译系统的质量。在你训练模型的过程中，同一示例句子在不同迭代轮次下的束搜索结果也被记录在TensorBoard中，可在*TEXT* 选项卡中查看（图3）。

记录的诊断信息包括具有以下字段的 json 文档：example_source（源句子 token）、example_target（真实目标句子 token）以及 hypotheses（与 beam size 为 10 的搜索结果相对应的 10 个假设）。请注意，

预测的翻译在神经机器翻译术语中通常被称为假设。
i.（2分）该模型在训练迭代过程中翻译质量是否有所提升？请给出该示例句子在第200次、第3000次以及最后一次迭代时的三个翻译示例来佐证你的回答。对于每次迭代，请选取第一个束搜索假设作为示例：

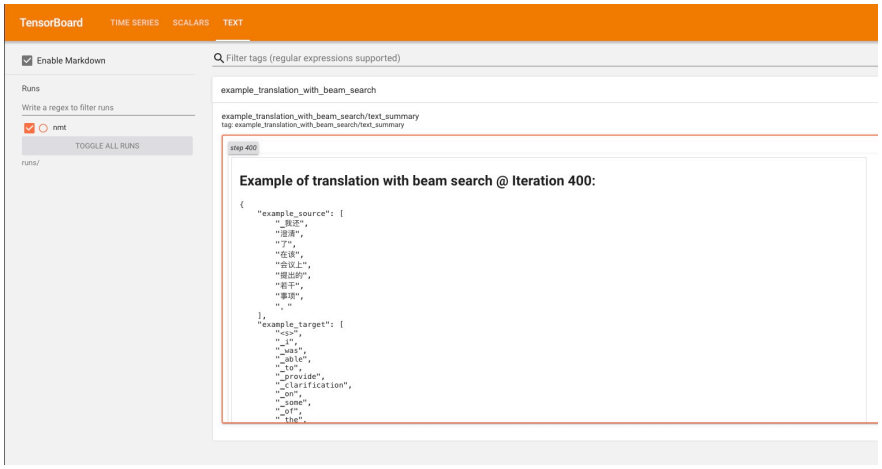


图 3：针对一个示例句子的束搜索翻译结果被记录在 tensorboard 中，涵盖多次迭代。相同数据可在工作目录下的 outputs/beam_search_diagnostics/ 文件夹中获取。

ii.（2 分）beam search 产生的各种假设在定性上如何比较？请再给出三个 beam search 在最后一次迭代中提出的假设示例来阐明你的答案。

提交说明

你需要在 GradeScope 上以两次提交的形式提交本作业——一次用于“Assignment 3 [coding]”，另一次用于“Assignment 3 [written]”：

1. 在云虚拟机上运行 collect_submission.sh 脚本，以生成你的 assignment3.zip 文件。请参阅 GCP 操作附录中的如何从云虚拟机下载 ***Gradescope*** 提交包。
2. 将你的 assignment3.zip 文件上传到 GradeScope 的“Assignment 3 [coding]”。
3. 将你的书面解答上传至 GradeScope 的“Assignment 3 [written]”。提交作业时，请务必按照 Gradescope 的提交说明为每道题的页面添加标签。若提交未正确添加标签，将会扣分。

附录：GCP 操作指南

如何获取 SSH 连接信息 按照 CS224n 的 GCP 指南中的说明操作后，你的 Google Cloud Compute Engine 控制台应如图 4所示。点击 SSH 旁边的箭头按钮，并选择 查看 *gcloud* 命令，如图所示。你将看到类似以下的内容。请记下区域、机器名称和项目名称，在示例中分别为 us-west4-b、nvidia-gpu-optimized-vmi-1-vm 和 grand-hangar-420500。**你的会有所不同！**

```
gcloud compute ssh --zone "us-west4-b" "nvidia-gpu-optimized-vmi-1-vm"  --
    ↪ project "grand-hangar-420500"
```

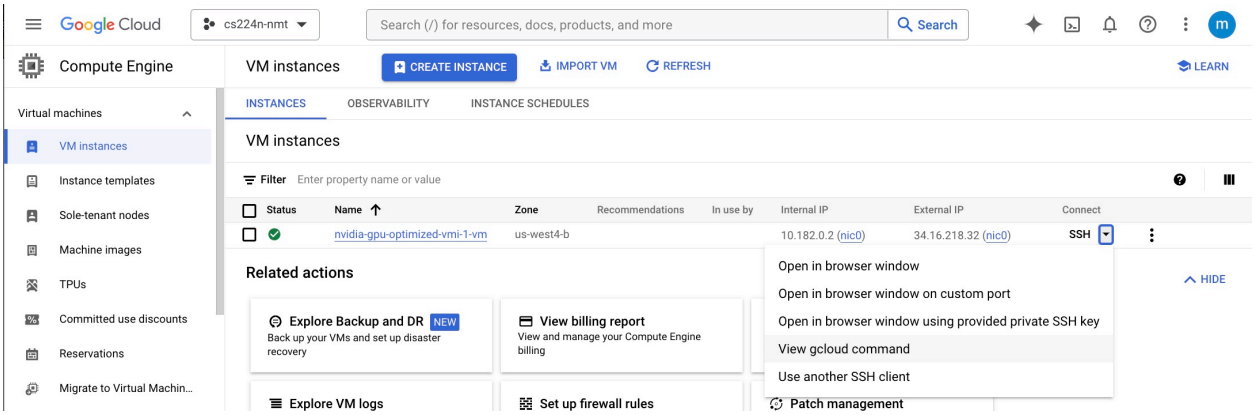


图 4：您创建的云虚拟机应基于 nvidia-gpu-optimized-vmi虚拟机镜像（VMI）。部署实例后，点击 SSH 附近的菜单并选择 查看 *gcloud* 命令 以获取 SSH 连接信息。

如何使用 SSH 隧道连接到云虚拟机 以下 ssh 连接命令包含 –ssh-flag ”-L 6007:localhost:6007” 选项，它还会将云机器的 6007 端口转发到您本地机器上的相同端口号（ssh 隧道）。因此，如果 tensorboard 服务器正在云虚拟机上运行并监听 6007 端口，在按如下方式建立 ssh 连接后，您将能够在本地机器的网页浏览器中通过 http://localhost:6007/ 访问。运行以下 ssh 命令以连接到云机器。 **请务必使用您自己的区域、项目名称和机器名称**

```
gcloud compute ssh --zone "us-west4-b" "nvidia-gpu-optimized-vmi-1-vm" --
→ project "grand-hangar-420500" --ssh-flag "-L␣6007:localhost:6007"
```

如何将代码从您的计算机复制到云虚拟机

- 压缩当前目录，该目录应包含 env-cpu.yml 和 env-gpu.yml 文件。

  ```
  zip -r student.zip *
  ```

- 将获得的 zip 文件复制到云机器

  ```
  gcloud compute scp student.zip  nvidia-gpu-optimized-vmi-1-vm:~/  --zone "
      → us-west4-b" --project "grand-hangar-420500"
  ```

如何在云虚拟机上训练 NMT 系统

- 如果您的云服务器上未安装 zip，请按以下方式安装后重试。

```
sudo apt-get install zip
```

- 在云机器上解压学生文件夹

```
unzip student.zip -d student
cd student
```

- 创建 conda GPU 环境

```
conda env create --file env-gpu.yml
```

- 激活 conda GPU 环境

```
conda activate cs224n-nmt-gpu
```

- 创建一个 tmux 会话，在其中启动机器翻译模型的训练

```
tmux new -s s-nmt

# start the training (in the tmux session)
sh run.sh train

# detach from the tmux session
CTRL+B D
```

- 创建一个 tmux 会话，在其中启动 TensorBoard

```
tmux new -s s-tboard

# start the tensorboard (in the tmux session)
tensorboard --logdir runs/ --port 6007

# detach from the tmux session
CTRL+B D
```

- 你可以根据需要附加到这两个 tmux 会话或从中分离。有关基本用例。
- 如果你使用适当的端口转发选项建立了 SSH 连接，你可以在本地机器的 Web 浏览器中通过以下地址查看 tensorboard http://localhost:6007/ （图 5）
- 训练完成后，运行测试阶段

```
tmux attach -t s-nmt
sh run.sh test
```

- 下一节展示如何使用在云虚拟机上生成的产物进行 GradeScope 提交。

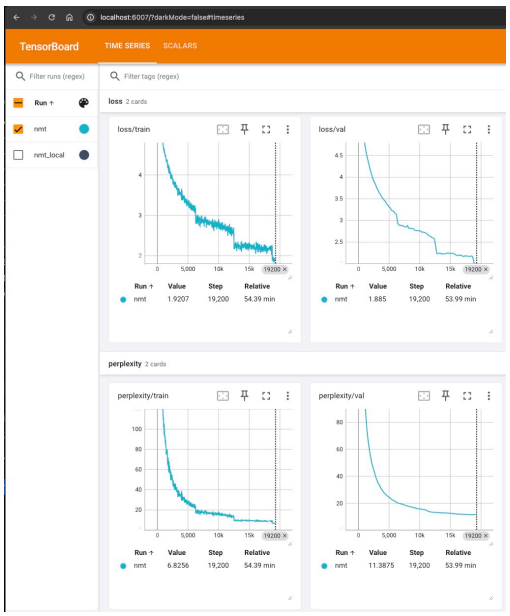


图 5：Tensorboard 在本地机器上运行，并显示云虚拟机上的损失值和困惑度值。

如何从云虚拟机下载 Gradescope 提交包

- 按上述说明连接到云虚拟机
- 在云虚拟机上，运行以下命令以创建 Gradescope 提交包（assign- ment3.zip）

```
sh collect_submission.sh
```

- 断开与云虚拟机的连接（通过运行 **exit**），或在本地机器上打开终端。
- 将 GradeScope 提交包下载到您的本地计算机

```
gcloud compute scp nvidia-gpu-optimized-vmi-1-vm:~/student/assignment3.zip
    → . --zone "us-west4-b" --project "grand-hangar-420500"
```

- 将 assignment3.zip 提交到 GradeScope

附录：Tmux 使用指南

tmux（终端复用器）是一个工具，可让你创建多个在后台运行的终端并在它们之间切换，即使你断开 ssh 会话后也能让它们继续运行。以下内容假设你已经（通过 ssh）连接到你的云虚拟机实例，并向你展示 tmux 的基本使用场景。要了解更多关于 tmux 的信息，请访问 https://github.com/tmux/tmux/wiki

- 创建并附加到一个新的 tmux 会话（命名为 *s-tboard*）

  ```
  tmux new -s s-tboard
  ```

- 在新会话中运行一个程序。我们将运行 tensorboard

  ```
  tensorboard --logdir runs/ --port 6007
  ```

- （在 tmux 会话内）从 tmux 会话分离，并让你启动的程序继续在会话中运行。

  ```
  CTRL+b d
  ```

- （从 tmux 分离后），查看所有 tmux 会话

  ```
  tmux ls
  ```

- 附加到特定的 tmux 会话（名为 s-nmt）

  ```
  tmux attach -t s-nmt
  ```

- 终止 tmux 会话（在会话内部操作）

  ```
  exit
  ```